# Comparing Agentic AI Frameworks

## for Java

Sandra Ahlgrimm
Senior Developer Advocate, GitHub

Timo Salm
Master Solutions Architect, Broadcom

November 2026

GitHub & vmware by Broadcom

# Who We Are



## Sandra Ahlgrimm

Senior Developer Advocate

**GitHub / Microsoft**

| | |
|---|---|
| **X** | @sKriemhild |
| **Bluesky** | @sandraahlgrimm.bsky.social |
| **LinkedIn** | linkedin.com/in/sandraahlgrimm |

JUG Berlin Brandenburg

Java Community Process

GraalVM Advisory Board



## Timo Salm

Master Solutions Architect

**Broadcom**

| | |
|---|---|
| **X** | @salmto |
| **Bluesky** | @timosalm.bsky.social |
| **LinkedIn** | linkedin.com/in/timosalm |

AMBASSADOR

Spring Certified Professional 2023

MVP Microsoft Most Valuable Professional

GitHub & vmware by Broadcom

# A Neutral Home for Agentic AI

The Agentic AI Foundation hosts ten open projects under the Linux Foundation

**Agentic AI Foundation**

*"The internet runs on open protocols and open source. So will agentic AI."*

**PLATINUM MEMBERS**

AWS, Anthropic, Block, Bloomberg, Cloudflare, Google, Microsoft, OpenAI

**MCP**
Connects agents to tools and data

**A2A**
Lets agents work together

**agentgateway**
Gateway for agents, models and tools

**Agent Skills**
AI gateway built on Envoy

**goose**
Open source AI agent for any LLM

**AGENTS.md**
Open format to guide coding agents

...

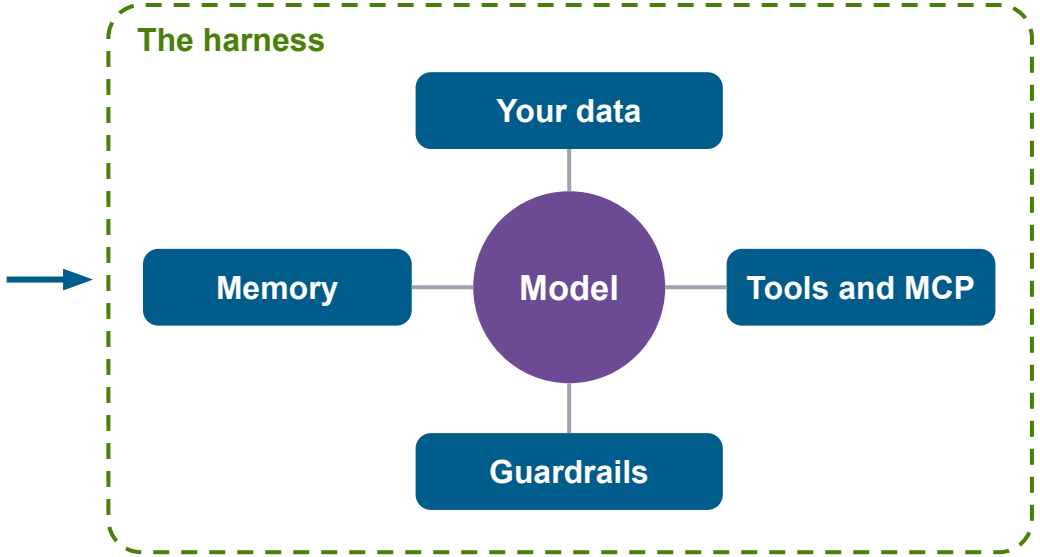
Highlighted projects come back later in this talk. aaif.io

# A Model Alone Is Not Enough

To be useful in your application, a model needs a harness

**On its own, a model**

- ✕ Makes up facts
- ✕ Forgets the conversation
- ✕ Cannot take action

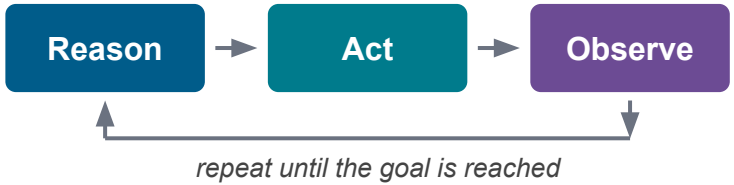



**Agentic AI frameworks build this harness.** They differ in how you program it.

# What Is Agentic AI?

From answering single questions to reaching goals on its own

An AI agent uses a model to decide its next step, acts through tools, and repeats this loop until the goal is reached.




*Agentic systems come in two forms. A workflow follows a path you define in code, while an agent chooses its own path.*

THE EVOLUTION OF LLM BASED SOLUTIONS

- **Standalone LLMs**
  Prompts in, text out for language tasks
- **Grounded Generation (RAG)**
  Answers based on your own business data
- **Tool Calling and MCP**
  The model can trigger actions in real systems
- **Agentic Systems**
  Workflows and agents built on models and tools

GitHub & vmware by Broadcom

# Two Approaches: Workflows and Agents

Who decides the next step, your code or the model?

## Workflows

Your code defines the path.

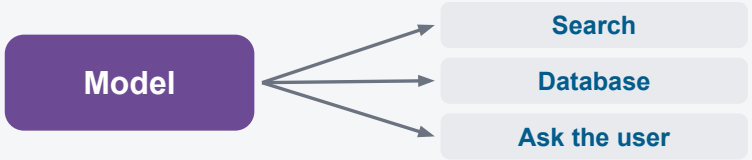



*Same steps on every run*

- ✓ Predictable, cheaper and easy to test
- ✗ Cannot adapt to the unexpected

## Pure Agents

The model decides the path.




*Picks tools in a loop until the goal is reached*

- ✓ Flexible for open ended tasks
- ✗ Higher cost, harder to control

**Start with a workflow for predictability.** Use an agent only when you truly need its flexibility.

# Workflow Patterns

Your code decides the steps. Pick the simplest pattern that fits the task

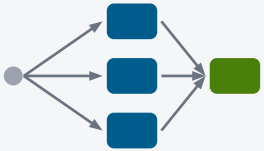
## Chain



Splits a task into steps. Each output feeds the next model call.

Used in the sample

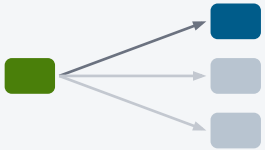
## Parallelization



Runs independent calls at the same time and combines the results.

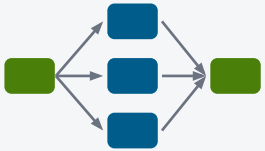
Used in the sample

## Routing



Classifies the input and sends it to a specialized prompt or model.

## Orchestrator-Workers



Splits the task at runtime and lets workers handle the parts.

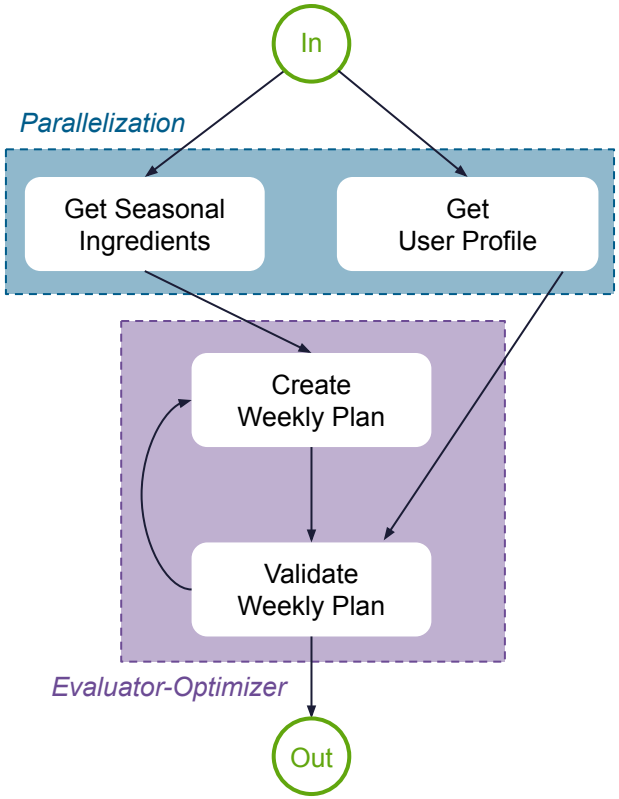
## Evaluator-Optimizer



Generates an answer, evaluates it, and retries with the feedback.

Used in the sample

# Sample Application




**AI Nutrition Planner** *powered by OpenAI (Gpt-5-nano)*

**Days & Meals**

| Monday All | Tuesday All | Wednesday All | Thursday All | Friday All | Saturday All | Sunday None |
|---|---|---|---|---|---|---|
| ☑ Breakfast | ☑ Breakfast | ☑ Breakfast | ☑ Breakfast | ☐ Breakfast | ☑ Breakfast | ☑ Breakfast |
| ☑ Lunch | ☐ Lunch | ☑ Lunch | ☐ Lunch | ☑ Lunch | ☐ Lunch | ☑ Lunch |
| ☑ Dinner | ☐ Dinner | ☑ Dinner | ☐ Dinner | ☐ Dinner | ☐ Dinner | ☑ Dinner |

Country (ISO code): DE

Additional instructions: Prefer quick recipes with preparation time under 30 minutes

Generate Plan

## Your Weekly Plan

### MONDAY

BREAKFAST

#### Wild Garlic Spinach Omelette 10 min

**Ingredients**

- 3 pieces Eggs
- 60 g Spinach
- 1/2 bunch Wild garlic
- 1 tbsp Chives
- 1 tsp Olive oil
- to taste Salt
- to taste Pepper

**Instructions**

Beat eggs with salt and pepper. Sauté wild garlic in oil, add spinach until just wilted. Pour in eggs and cook gently, folding until set. Finish with chopped chives.

| NUTRITION | |
|---|---|
| Calories | 270 kcal |
| Protein | 18,0 g |
| Carbs | 4,0 g |
| Fat | 19,0 g |
| Sodium | 280 mg |

LUNCH

# Meet the Frameworks

All three compared in Spring Boot 4 applications

Spring AI

Official Spring project

Aligns with Spring ecosystem design principles

Enterprise support available



LangChain4j

Initiated in early 2023 due to a lack of Java counterparts to Python libraries like LangChain, LlamaIndex

Experimental Agentic module for workflows and agents

Spring Boot starter, also Quarkus and more



Embabel

Announced in 2025 to bring Agentic AI to enterprise JVM applications

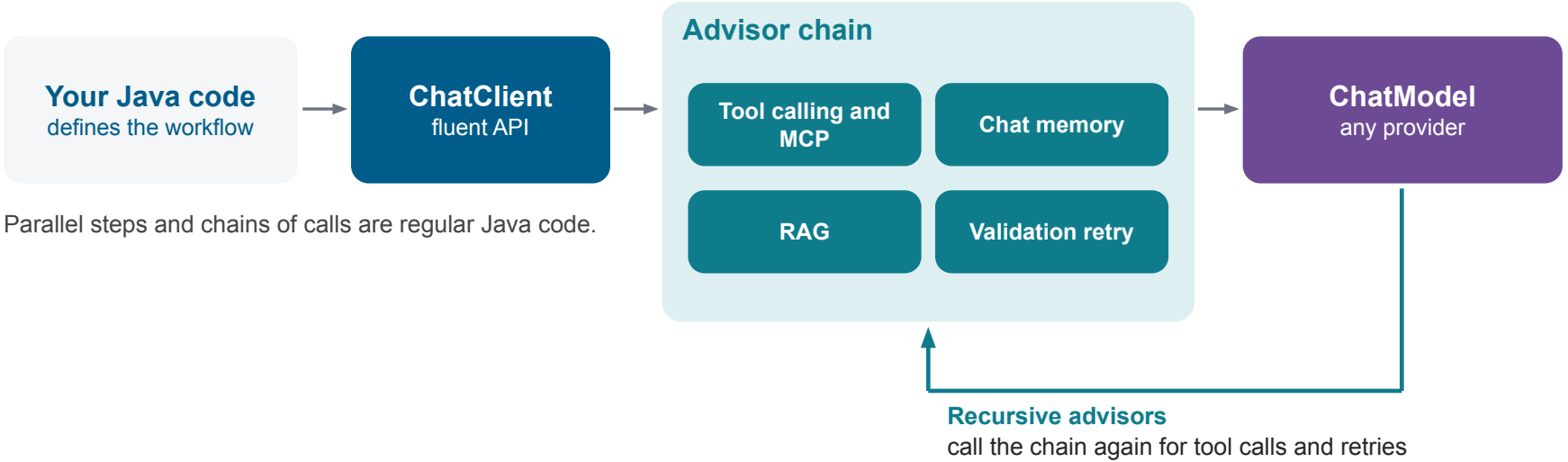
Founded by Spring creator Rod Johnson

Built on Spring AI

More agent frameworks on the JVM: GitHub Copilot SDK, Koog by JetBrains, Google ADK for Java and others.

# Spring AI: Core Concepts

A fluent client and a chain of advisors. Orchestration is plain Java




Parallel steps and chains of calls are regular Java code.

**You write the flow.** Advisors add tools, memory, RAG and loops to every call.

# Spring AI: The ChatClient in Action

One fluent call creates the weekly plan in the sample

```
WeeklyPlan plan = chatClient.prompt()
    .system(RECIPE_CURATOR)
    .user(u -> u.text(PROMPT).params(inputs))
    .advisors(validationRetryAdvisor)
    .tools(askUserQuestionTool)
    .call()
    .entity(WeeklyPlan.class);
```

1. **Advisors**
   A recursive advisor runs the validation loop.
2. **Tools**
   The model can act and ask the user.
3. **Structured output**
   The answer becomes a Java record.

Agent capabilities such as skills, todo lists, user questions and subagents come from the experimental project **`spring-ai-agent-utils`**.

# Spring AI: A Recursive Advisor

The ValidationRetryAdvisor implements evaluator-optimizer in a few lines

```
@Override
class ValidationRetryAdvisor implements CallAdvisor {

  public ChatClientResponse adviseCall(ChatClientRequest request,
                                       CallAdvisorChain chain) {
    var response = chain.nextCall(request);
    for (int i = 0; i < MAX_RETRIES && !isValid(response); i++) {
      request = withFeedback(request, response);       // add plan and feedback
      response = chain.copy(this).nextCall(request);   // call the rest again
    }
    return response;
  }
}
```

vmware by Broadcom

Broadcom Proprietary and Confidential. Copyright © 2025 Broadcom.
All Rights Reserved. The term “Broadcom” refers to Broadcom Inc. and/or its subsidiaries.

spring AI

# Code Sample

Core Concepts & Workflow Patterns



GitHub & vmware by Broadcom

# LangChain4j: Core Concepts



AI services: declare an interface, and LangChain4j implements it

## Spring Boot creates the bean

```
@AiService
interface NutritionExpert {
  @SystemMessage("Act as a nutritionist")
  String chat(String question);
}
```

## Or build it yourself

```
NutritionExpert expert = AiServices
    .builder(NutritionExpert.class)
    .chatModel(chatModel)
    .tools(tools)
    .build();
```

**AI services handle for you**

Structured output | Tools | Chat memory | RAG | Streaming

**Low level abstraction:** `ChatModel.chat(messages)` gives full control, but you handle prompts, memory and tools yourself.

# LangChain4j: The Agent Abstraction



An agent is an AI service that can be composed, so you declare the flow

```
interface SeasonalIngredientAgent {

  @UserMessage("Which food is in season in {{country}}?")
  @Agent(description = "Finds seasonal food",
         typedOutputKey = SeasonalIngredients.class)
  SeasonalIngredients fetch(@V("country") String country);
}
```

**1** **Description**

A supervisor reads it to pick the right agent at runtime.

**2** **Output key and AgenticScope**

Results go to shared state. Later agents read them by name.

**A** **Workflow agents**

Sequence, parallel, loop and conditional agents compose agents.

**B** **Pure agentic patterns**

A supervisor or a goal oriented planner decides at runtime.

The `langchain4j-agentic` module is still marked experimental.

GitHub & vmware by Broadcom

# LangChain4j: Agents in the Sample



A parallel agent for the inputs, and a loop agent with an exit condition

### A parallel agent fetches the inputs

```
@ParallelAgent(subAgents = {
    UserProfileAgent.class,
    SeasonalIngredientAgent.class })
void fetchPlanInputs();

@ParallelExecutor
static Executor executor() { ... }
```

### A loop agent with an exit condition

```
@LoopAgent(maxIterations = 3,
  subAgents = { WeeklyPlanCreator.class,
                NutritionGuard.class })
WeeklyPlan createValidatedWeeklyPlan();

@ExitCondition
static boolean exit(ValidationResult r) {
  return r.allPassed();
}
```

Both run as steps of a `@SequenceAgent`. `AgenticServices.createAgenticSystem(...)` creates the whole workflow.

GitHub & vmware by Broadcom



LangChain4j

# Code Sample

Core Concepts & Workflow Patterns



GitHub & vmware by Broadcom

# Embabel: Core Concepts

Agents are classes with actions and goals, and every model call is typed



```
@Agent(description = "Supports meal planning")
class NutritionPlannerAgent {
  @Action
  SeasonalIngredients fetchSeasonalIngredients(WeeklyPlanRequest request) { ... }

  @AchievesGoal(description = "Provides a plan")
  @Action
  WeeklyPlan createWeeklyPlan(SeasonalIngredients ingredients, Ai ai) {
    return ai.withAutoLlm().createObject(prompt, WeeklyPlan.class);
  }
}
```

**The planner finds the flow.** No sequence is coded. Order, parallel steps and loops follow from the types.



Embabel

# Code Sample

Core Concepts & Workflow Patterns



GitHub & vmware by Broadcom

# MCP and A2A: Open Protocols for Agents

MCP connects an agent to tools and data. A2A connects an agent to other agents

## MCP Model Context Protocol

**Provides** Tools | Resources | Prompts

*over stdio or Streamable HTTP*

| | Client | Server |
|---|---|---|
| Spring AI | Built in | @McpTool, etc. |
| LangChain4j | Built in | stdio, community |
| Embabel | Via Spring AI | Via Goals and @Export(remote = true) |

## A2A Agent2Agent Protocol

**Elements** Agent Card | Tasks | Messages | Artifacts

*over HTTP, with streaming and push notifications*

| | Client | Server |
|---|---|---|
| Spring AI | A2A Java SDK | Experimental |
| LangChain4j | Built in | Not provided |
| Embabel | A2A Java SDK | Experimental |

**Build a tool or an agent once, use it everywhere.** Both protocols are open and vendor neutral.

Built in | Experimental, or some effort | Limited

GitHub & vmware by Broadcom

# MCP Client Support

All three can connect to MCP servers, but each one sets up the client in its own way.

**Spring AI**

Built in

```
spring.ai.mcp.client.streamable-http.connections.recipes.url=...
chatClient.prompt(question).toolCallbacks(mcpTools).call();
```

**LangChain4j**

Built in

```
var transport = StreamableHttpMcpTransport.builder().url(...).build();
var client = DefaultMcpClient.builder().transport(transport).build();
var tools = McpToolProvider.builder().mcpClients(client).build();
AiServices.builder(Planner.class).toolProvider(tools).build();
```

**Embabel**

Via Spring AI

```
// Spring AI client configuration, MCP tools grouped as tool groups
@Bean McpToolGroup recipes(List<McpSyncClient> clients) { ... }
@Action(toolGroups = "recipes")
WeeklyPlan createWeeklyPlan(WeeklyPlanRequest request, Ai ai) { ... }
```

# MCP Server Support: Spring AI vs. Embabel

Spring AI publishes annotated methods. Embabel publishes whole goals of an agent

## Spring AI

Built in

```
@McpTool(description = "Weekly plan")
WeeklyPlan createNutritionPlan(
    WeeklyPlanRequest request) { ... }

@McpResource(uri = "profiles://{name}")
UserProfile profile(String name) { ... }

@McpPrompt(name = "plan-my-week") ...
```

## Embabel

Built in

```
@AchievesGoal(
    description = "Provides a plan",
    export = @Export(remote = true))
@Action
WeeklyPlan createNutritionPlan() { ... }
```

**Expose a tool once, use it in any MCP client.** Embabel runs every action needed to reach an exported goal.

# Patterns for a Single Agent

Harness features that keep one agent focused and under control

## Agent Skills

Instructions and scripts the agent loads only when needed.

| | |
|---|---|
| Spring AI | `SkillsTool` |
| LangChain4j | `Skills` |
| Embabel | `Skills` |

## Tool Search

The agent loads only the tool definitions it needs.

| | |
|---|---|
| Spring AI | `ToolSearchToolCallingAdvisor` |
| LangChain4j | `ToolSearchStrategy` |
| Embabel | `@UnfoldingTools` |

## Plan and Execute

The agent plans its steps first, then works through them.

| | |
|---|---|
| Spring AI | `Different approach (TodoWriteTool)` |
| LangChain4j | `GOAP planner` |
| Embabel | `GOAP planner` |

## Human in the Loop

The agent asks for input or approval before it goes on.

| | |
|---|---|
| Spring AI | `AskUserQuestionTool` |
| LangChain4j | `@HumanInTheLoop` |
| Embabel | `WaitFor` |

Built in | Experimental, or some effort | Limited

# Multi-Agent Patterns

Keep one agent in charge, and add more only where it pays off

## LLM as a Judge

A second model rates the result and sends feedback until it passes.

| | |
|---|---|
| **Spring AI** | `via Recursive advisor` |
| **LangChain4j** | `@LoopAgent` |
| **Embabel** | `@State loop` |

## Subagents

The main agent delegates a task to an agent with its own context.

| | |
|---|---|
| **Spring AI** | `TaskTool` |
| **LangChain4j** | `Agents are smaller building blocks` |
| **Embabel** | `Subagent tool or RunSubagent` |

Built in | Experimental, or some effort | Limited

GitHub & vmware by Broadcom

spring AI

Embabel

LangChain4j

# Code Samples

Agentic Patterns



GitHub & vmware by Broadcom

# Comparing Capabilities for Spring Developers

Our assessment, based on the sample application in November 2026

| Capability | Spring AI | LangChain4j | Embabel |
|---|---|---|---|
| **Deterministic workflows** | Advisors & plain Java code | Workflow agents | @State and planner |
| **Pure agents** | Tool calling loop | Supervisor agent | Built in |
| **Memory and state** | ChatMemory advisors | AgenticScope | Blackboard |
| **Agentic patterns** | Experimental | Built in, some experimental | Built in, some experimental |
| **Observability** | Model level only | Agent listeners | Agent level |
| **Testing** | Mocks, evaluators | Mostly manual | Advanced Testing support |
| **MCP** | Client and server | No HTTP server support | Client and server |
| **A2A** | Experimental | Client only | Experimental |
| **Maturity** | Agentic experimental, official Spring project, enterprise support | Agentic experimental | GA, plus some experimental features |

Built in | Experimental, or some effort | Limited

GitHub & vmware by Broadcom

# Choosing the Right Framework

There is no single winner. Match the programming model to your problem

Spring AI

**Choose it for**

Business-critical workloads that require enterprise-grade support

Mostly deterministic workflows with a few agentic steps

Full control and the Spring ecosystem



LangChain4j

**Choose it for**

Ready made workflow and agent building blocks

Also on Quarkus, Micronaut, Helidon, Jakarta EE



Embabel

**Choose it for**

Rich domain models that should drive the flow

Flows that adapt at runtime, with testable actions

**You can combine them.** Embabel runs on Spring AI, and MCP and A2A connect agents across frameworks.

# Learn Spring AI

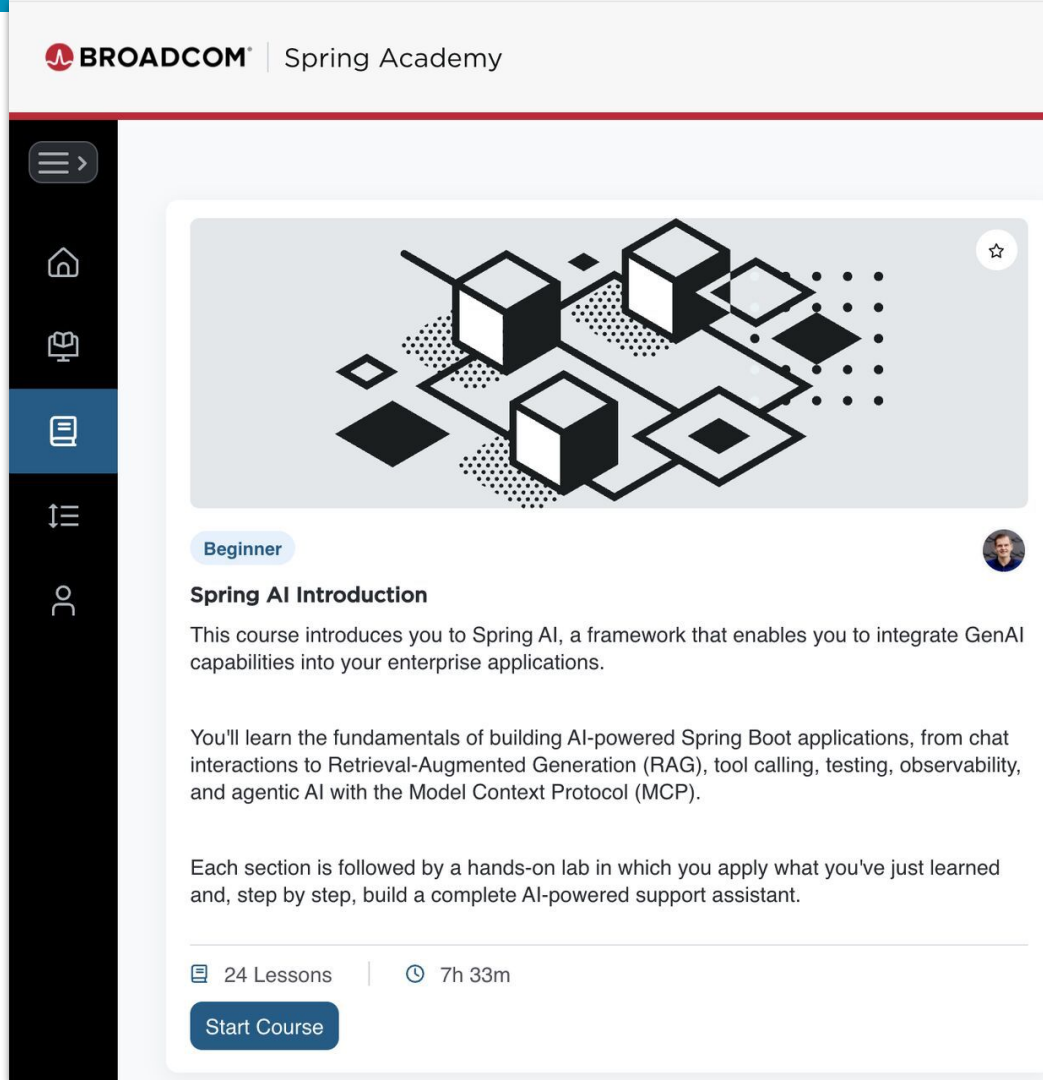
With Timo's free course on Spring Academy

Build AI applications and agents step by step, with hands-on labs.

*Free of charge. You only need to register.*

spring.academy/courses/spring-ai-intro

GitHub & vmware by Broadcom



Follow us
@sKriemhild
@salmto

# Thank You

https://github.com/SandraAhlgrimm/ai-nutrition-planner

GitHub & vmware by Broadcom